

Migration plan

What ships in week 1, month 1 and quarter 1 — no big-bang, no code freeze

Rules this plan follows

1. **No big-bang.** Nothing here needs a moment where the old system stops and the new one starts.
2. **Every step ships on its own.** If the plan is cancelled in any given week, what shipped so far still has value and nothing is left half-migrated.
3. **Every step has a written rollback**, rehearsed for the risky ones.
4. **Additive before subtractive.** Add the new path, move traffic to it, delete the old path a week later once it has survived a billing cycle.
5. **Product work continues** at about 70% of capacity. A plan that stops the business gets cancelled in week three.

Rescue work is roughly 30% of engineering time — about six engineer-days a week. Deliberately modest, because a plan needing 60% gets negotiated down to 20% in month two anyway.

Billing days are the first week of the month; nothing risky ships in that window. Peak is November, so everything structural lands before October.

Week 1 — stop the bleeding

Goal: close the two live exposures, and be able to see and undo what we do. No architecture changes, no refactoring, no route handler touched.

DAY	SHIPS	VERIFIED BY	ROLLBACK
Mon	Error tracking and uptime alerting	Throw in a canary route; alert fires within 60s	Remove the SDK; it is additive
Mon	Inventory of every consumer of the database credential	Reviewed with the team	Document only
Tue	CI pipeline: build, deploy, one-command rollback	Push a no-op change through it, then roll it back	Laptop deploy stays available all week
Wed	RLS in log-only mode on all tables	24h of "would have denied" logs, reviewed	One setting, off
Thu	RLS enforced on <code>customers</code> , then <code>subscriptions</code> , then <code>orders</code>	Account pages walked per table; denial log empty	Per table, one at a time
Thu 04:00	Stripe key rotated via the dual-key window	Test charge on the new key; old key traffic hits zero	Old key live for 24h

DAY	SHIPS	VERIFIED BY	ROLLBACK
Fri 04:00	Database password rotated, secrets moved to a managed store	Smoke test every consumer from Monday's inventory	Old password restorable; rehearsed Thursday
Fri	Git history purged, push-time secret scanning on	Scanner blocks a planted fake key	Advisory for 48h before it blocks

Done when: no customer can read another customer's row; no credential in the repo works; an incident pages someone within a minute; anyone on the team can deploy and roll back.

Biggest risk: the Friday rotation is the one step that can take the site down. Mitigations are the Monday inventory, a Thursday rehearsal against a copy of the VPS, a 04:00 window, two people awake, and the old password held for 24 hours before it is revoked.

Month 1 — build the safety net, then make the first cut

Weeks 2–3

Characterization tests on the three money paths — checkout, plan change, cancellation. These pin down what the system does *today*, bugs included. Where behaviour looks wrong, the test records the wrong behaviour and gets a linked ticket; changing it during a refactor would smuggle a business decision into a technical change.

I write the first path in a pairing session with whoever owns billing. They write the second with me watching, and the third alone.

A staging environment from a sanitised production dump — real data volumes, scrambled personal details. Without it, "we tested it" means "it worked on a laptop with eleven rows".

Individual admin accounts, shared login disabled. One day of work; closes finding 7.

Node 18, then 20, security advisories only.

Week 4

Transactions and idempotency on the plan-change path — the [worked refactor](#) — shipped behind a flag:

1. New service path added alongside the old handler, flag off.
2. On for internal accounts only. We change our own plans and deliberately break things.
3. On for 5% of traffic for 48 hours, watching error rate and payment-failure rate.
4. Ramped to 100% over three days.
5. Old handler deleted a week later, after a billing cycle has run through the new one.

Rollback is flipping the flag — not a revert, not a redeploy — because at 3am on a billing day the only acceptable rollback is one a tired person cannot get wrong.

A nightly reconciliation job comparing our subscription rows against Stripe and reporting drift into Slack. It turns finding 6 from an invisible problem into a number we can watch fall, and it keeps working as a safety net long after the code is fixed.

Done when: the three money paths have tests running in CI on every push; plan changes are transactional and idempotent at 100%; drift is reported nightly and trending to zero; every engineer has deployed through CI at least once.

Quarter 1 — get the logic out of the route handlers

The same five steps per domain — pricing, subscriptions, orders, inventory:

1. Characterization tests around the existing behaviour.
2. Create the service module, move the logic, leave the handler doing transport only.
3. Point one caller at it behind a flag, ramp.
4. Point the remaining callers at it, including the mobile app's endpoint.
5. Delete the duplicated logic. **Not optional, not "later"** — a strangler that skips the deletion leaves two implementations, which is worse than one bad one.

One module per fortnight: six sprints, four modules, two sprints of slack. There will be an incident and someone will be on holiday, and a plan with no slack fails in week three.

Also in the quarter:

- **Frontend direct-database burn-down.** 31 direct Supabase calls today. The number goes on a dashboard and only goes down; each becomes an API endpoint as its module is extracted.
- **Expand-and-contract schema changes**, so no migration needs a maintenance window: add the column, write both, backfill, read the new one, stop writing the old, drop it.
- **A coverage ratchet, not a target.** Coverage on changed lines may not fall. No number is set and nobody backfills tests for code they are not touching.
- **The standards land** in the sequence in the [standards proposal](#).
- **A performance pass in October**, before peak: the N+1 on the orders page, missing indexes on `subscriptions(user_id)` and `orders(created_at)`, and a load test at three times December's peak.

Done when: pricing has one implementation and the mobile app uses it; direct database calls from the browser are under five; every money path is transactional, idempotent and covered; the reconciliation job reports zero drift for four straight weeks.

Not in this plan

NOT DOING	WHY
Rewrite	A year of no product delivery to end up where we are, minus the bugs we forgot to reimplement
Framework migration	Solves none of the nine findings
Microservices	Four engineers. This is a distribution problem we do not have
80% coverage mandate	Produces tests written to hit a number. The ratchet produces tests that catch bugs
Backfilling tests everywhere	Test what you touch. Untouched code is not where the risk is
Replacing Supabase	It is fine. It was configured badly, and that was fixed in week 1

How we know it is working

MEASURE	TODAY	AT 90 DAYS
Change failure rate	~1 in 4 deploys needs a follow-up fix	Under 1 in 10
Time to detect an incident	~3 hours	Under 5 minutes
Billing drift found by reconciliation	Unknown — invisible to us	Zero, four weeks running
Direct database calls from the browser	31	Under 5

Not measured: lines refactored, test count, coverage percentage. Those measure activity.

What could derail it

- **An incident eats a sprint.** Expected — that is what the two slack sprints are for. We drop scope, never sequence.
- **The business needs a big feature in month 2.** Rescue work drops to 15% for that sprint and the quarter extends. What does not happen is skipping the safety net to land the feature faster, because that trade is how this situation was built.
- **RLS breaks an account page nobody thought about.** The most likely week-1 failure, which is why log-only mode runs first and each table enforces separately.
- **The team does not adopt the practices.** The real risk, and the subject of the [standards proposal](#).